Bézier Curve — Algebraic Expansion

Code Explanation for Julia Beginners

`01-bezier-algebra.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains the Julia code in `01-bezier-algebra.jl` for readers who are new to Julia. It focuses on the parts of the language that may look unfamiliar: type annotations, constructors, operator overriding, multiple dispatch, and how data flows through the print functions.

Contents

1 Type Map: How the Pieces Fit Together **2**

2 The `Poly` Struct and Its Constructor **3**
2.1 The struct itself . . . 3
2.2 The convenience constructor . . . 3

3 Operator Overriding **5**
3.1 What is an operator? . . . 5
3.2 Why override? . . . 5
3.3 The two multiplication methods . . . 5
3.3.1 First method: `Poly * number` . . . 5
3.3.2 Second method: `number * Poly` . . . 5
3.4 Multiple dispatch — how Julia chooses which method to call . . . 6

4 Polynomial Addition **6**

5 Polynomial Evaluation via Horner's Method **7**
5.1 Callable structs . . . 7
5.2 Horner's method . . . 7

6 The `expand_bernstein` Function and Pascal's Triangle **8**
6.1 What is `binomial(n, k)`? 8
6.2 Pascal's Triangle — the Cheat Sheet Nobody Printed 8
6.3 How `expand_bernstein` Uses `binomial` 9

7 The `BezierExpansion` Struct — A Concrete Map **11**

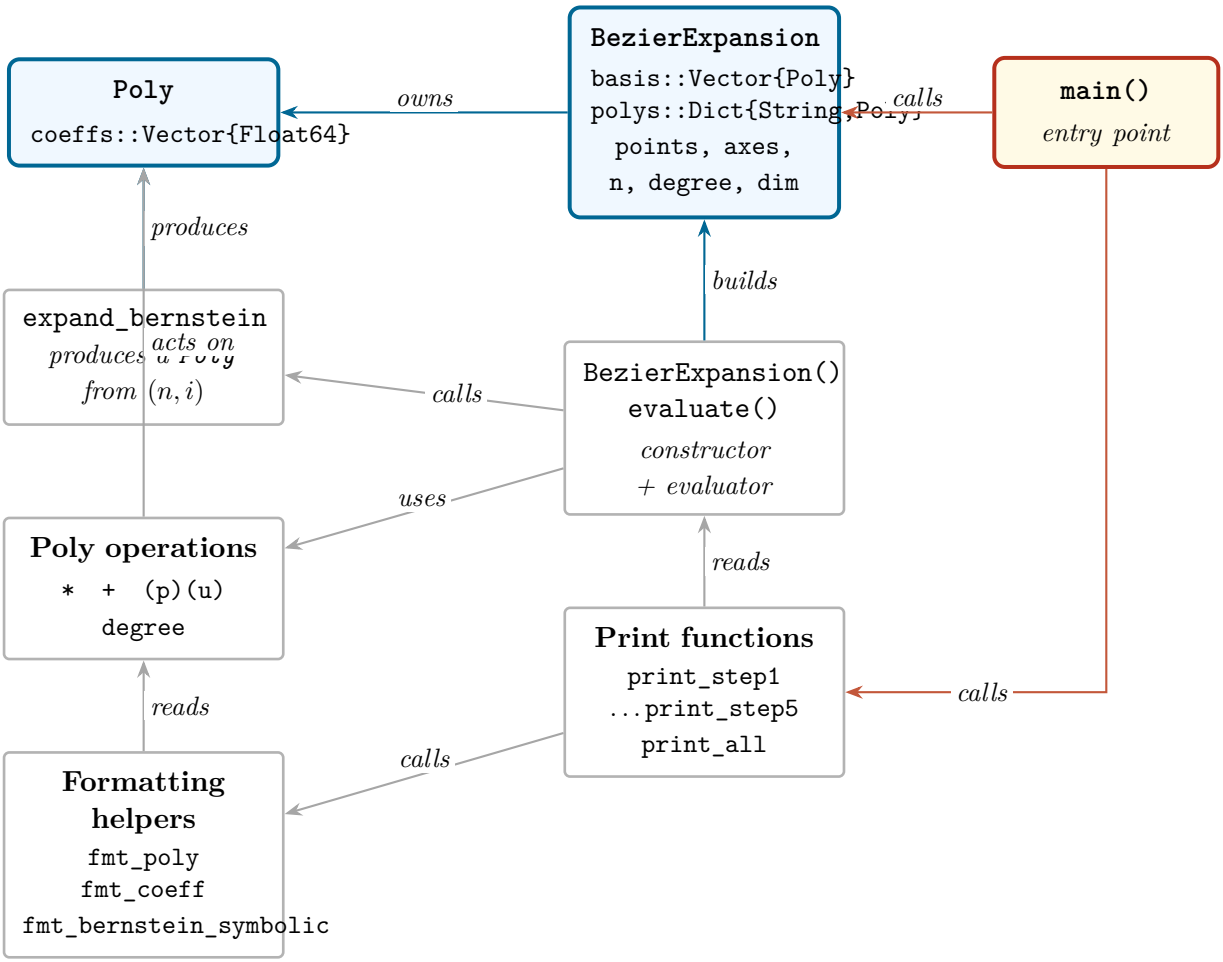
8 The Print Functions **12**
8.1 `print_step1_bernstein_form(exp)` 12
8.2 `print_step2_substitution(exp)` 13
8.3 `print_step3_expanded_terms(exp)` 14
8.4 `print_step4_collected(exp)` 15
8.5 `print_step5_evaluation(exp, u_values)` 15

9 Summary: How Data Flows Through the Program **17**

1. Type Map: How the Pieces Fit Together

Before diving into individual functions, it helps to see the whole picture at once. Julia has no classes, but the code has a clear object-like structure: two structs, a cluster of free functions that operate on them, and one entry point that ties everything together.

The diagram below shows every type and function group, and the arrows show who calls, owns, or produces whom.



Reading the diagram:

Arrow colour	Meaning
Blue	Structural relationship — one type owns or produces another
Grey	Function call or data dependency
Red	Entry point — `main()` triggering the whole chain

The two **blue boxes** are structs (Julia types). Everything else is a free function or function group. `Poly` is the atom — everything else is built from it or depends on

> it. `BezierExpansion` is the molecule — it assembles `Poly` objects into a complete curve. The print and formatting functions sit on top and never modify any data, they only read and display.

2. The `Poly` Struct and Its Constructor

2.1. The struct itself

```
struct Poly
    coeffs::Vector{Float64}
end
```

This defines a new data type called `Poly`. It has one field: `coeffs`, which must be a `VectorFloat64` — a list of 64-bit floating-point numbers.

To create a `Poly` you would write:

```
p = Poly([1.0, -2.0, 1.0])
```

That works fine. But if you try to pass integers:

```
p = Poly([1, -2, 1])    # ERROR -- these are Int64, not Float64
```

Julia would refuse because the field type is `VectorFloat64` and `[1, -2, 1]` is `VectorInt64`. That is why the convenience constructor below exists.

2.2. The convenience constructor

```
Poly(c::Vector{<:Real}) = Poly(Float64.(c))
```

This looks like a function, but it has the same name as the type (`Poly`), so Julia treats it as an **outer constructor** — a factory function that builds a `Poly`.

Breaking it down piece by piece:

`c::Vector{<:Real}`

This is a **type annotation** on the argument `c`. It says: "`c` must be a vector, and the elements can be *any subtype of* `Real`."

`Real` is Julia's abstract type for all real numbers. Its subtypes include `Int64`, `Float32`, `Float64`, `Rational`, and many others. The `<:` symbol means "is a subtype of". So `Vector{<:Real}` means "a vector of any kind of real number."

> **What counts as `Vector{<:Real}`?**
>
> | Value | Accepted? |
> |---|---|
> | `[1, 2, 3]` | Yes — `Vector{Int64}`, and `Int64 <: Real` |
> | `[1.0, 2.0, 3.0]` | Yes — `Vector{Float64}`, and `Float64 <: Real` |
> | `[1//2, 3//4]` | Yes — `Vector{Rational}`, and `Rational <: Real` |
> | `["a", "b"]` | No — `String` is not a subtype of `Real` |

`Float64.(c)`

The dot (`.`) before the parentheses is Julia's **broadcast** syntax. It means: "apply the function `Float64()` to *every element* of `c`."

> `Float64.([1, -2, 1])` → `[1.0, -2.0, 1.0]`
>
> Each integer is individually converted to a `Float64`.

So the full line reads: "If you give me a vector of any real numbers, I will convert every element to `Float64` and hand that to the real `Poly` constructor."

This is called a **sanity constructor** — it makes the type convenient to use without forcing the caller to think about numeric types.

> **Why not just store integers directly?**
>
> The struct uses `Float64` for all arithmetic that follows (polynomial evaluation, Horner's method, etc.). Mixing integer and float arithmetic in Julia can be surprising, so converting everything to `Float64` at construction time keeps the rest of the code simple and predictable.

Summary: `Poly` Functions

All functions associated with `Poly`:

Function	Signature	Returns	Purpose
Constructor	`Poly(c::Vector{<:Real})`	`Poly`	Convert any numeric vector to `Poly{Float64}`
`*`	`(Poly, Real)` or `(Real, Poly)`	`Poly`	Scale every coefficient by a number
`+`	`(Poly, Poly)`	`Poly`	Add two polynomials term by term
`(p)(u)`	`(Poly, Real)`	`Float64`	Evaluate polynomial at u via Horner
`degree`	`(Poly)`	`Int`	Return `length(coeffs) - 1`

In an OOP language these would be methods of a `Poly` class. In Julia they are standalone functions that happen to take a `Poly` as their first (or only) argument. Multiple dispatch handles the rest.

3. Operator Overriding

3.1. What is an operator?

Operators like `*`, `+`, `-` look special, but in Julia they are just ordinary **functions** with short names. When you write `3 * 5`, Julia is actually calling a function named `*` with arguments `3` and `5`.

These built-in functions live in a module called `Base`, which is Julia's standard library. So the full name of the multiplication function is `Base.:*`.

The `.:` syntax: the dot accesses something inside a module (like `Base.something`), and the colon `:` is needed because `*` is a special symbol, not a plain identifier. Without the colon, Julia would try to parse `Base.*` as invalid syntax.

3.2. Why override?

Julia already knows how to multiply two numbers: `3 * 5 = 15`. But it has no idea what `Poly([1.0, 2.0]) * 3` should mean — you invented the `Poly` type, so you must teach Julia the rule.

Operator overriding means adding a new method to an existing operator function, so it knows what to do with your custom type. You are not replacing the original `*` — you are *extending* it with an extra case.

3.3. The two multiplication methods

```julia
# Scalar multiplication
Base.:*(p::Poly, s::Real) = Poly(p.coeffs .* s)
Base.:*(s::Real, p::Poly) = p * s
```

3.3.1. First method: `Poly * number`

```julia
Base.:*(p::Poly, s::Real) = Poly(p.coeffs .* s)
```

This teaches Julia what to do when the **left** argument is a `Poly` and the **right** argument is any real number.

`p.coeffs .* s` uses broadcast again: multiply every coefficient in the vector by the scalar `s`. The result is wrapped in a new `Poly`.

```julia
Poly([1.0, -2.0, 1.0]) * 3
```

`p.coeffs .* s` $= [1.0, -2.0, 1.0] \,.\times\, 3 = [3.0, -6.0, 3.0]$

Returns `Poly([3.0, -6.0, 3.0])`, representing $3 - 6u + 3u^2$.

3.3.2. Second method: `number * Poly`

```
Base.:*(s::Real, p::Poly) = p * s
```

This teaches Julia what to do when the **left** argument is a number and the **right** argument is a `Poly`.

The body is simply `p * s` — it flips the order and calls the first method. No logic is duplicated; we just say "this is the same as the other way around."

> Without this second method, writing `3 * poly` would fail. Julia does not assume that `*` is commutative — it only knows what you explicitly tell it. For types where order matters (like matrix multiplication), `A * B` and `B * A` can be completely different operations, so Julia is right to keep them separate.

3.4. Multiple dispatch — how Julia chooses which method to call

When you write `a * b`, Julia looks at the types of **both** `a` and `b` together and finds the method whose type signature is the best match. This is called **multiple dispatch**.

Three calls, three different methods:

Expression	Types	Method chosen
`3 * 5`	`(Int64, Int64)`	built-in numeric `*`
`poly * 3`	`(Poly, Int64)`	first method above
`3 * poly`	`(Int64, Poly)`	second method above

Compare with Python or Java (single dispatch): In those languages, `a * b` asks only the type of `a`: "do *you* know how to multiply yourself by `b`?" The type of `b` is handled as a secondary concern inside that method. Julia asks about both arguments simultaneously and dispatches to the most specific matching method.

This is why the two methods `(Poly, Real)` and `(Real, Poly)` are genuinely separate in Julia — they have different type signatures and Julia treats them as distinct choices.

4. Polynomial Addition

```
function Base.:+(a::Poly, b::Poly)
    n = max(length(a.coeffs), length(b.coeffs))
    ca = vcat(a.coeffs, zeros(n - length(a.coeffs)))
    cb = vcat(b.coeffs, zeros(n - length(b.coeffs)))
    Poly(ca .+ cb)
end
```

This overrides `+` for the case where **both** arguments are `Poly` — another example of multiple dispatch, this time with a two-argument signature `(Poly, Poly)`.

The zero-padding problem. Two polynomials can have different degrees, meaning their `coeffs` vectors have different lengths. You cannot add two vectors of different lengths element-by-element.

The fix:

1. Find the longer length `n`.
2. Extend the shorter vector with zeros at the end (`vcat` concatenates two vectors; `zeros(k)` makes a vector of `k` zeros).
3. Add element-by-element with `.+` (broadcast addition).

Adding $3 - 6u + 3u^2$ and $1 + 2u$:

`a.coeffs = [3.0, -6.0, 3.0]` (length 3)
`b.coeffs = [1.0, 2.0]` (length 2)

`n = max(3, 2) = 3`

Pad `b`: `vcat([1.0, 2.0], zeros(1)) = [1.0, 2.0, 0.0]`

Add: `[3.0, -6.0, 3.0] .+ [1.0, 2.0, 0.0] = [4.0, -4.0, 3.0]`

Result: `Poly([4.0, -4.0, 3.0])` $= 4 - 4u + 3u^2$

5. Polynomial Evaluation via Horner's Method

```julia
function (p::Poly)(u::Real)
    result = 0.0
    for c in reverse(p.coeffs)
        result = result * u + c
    end
    result
end
```

5.1. Callable structs

The function signature `(p::Poly)(u::Real)` looks unusual. It means: "make a `Poly` object callable like a function."

After this definition, if `p` is a `Poly`, you can write `p(0.5)` and Julia will call this method, passing `p` as the struct and `0.5` as `u`. This is called a **functor** — an object that behaves like a function.

5.2. Horner's method

To evaluate $c_0 + c_1u + c_2u^2 + c_3u^3$, the naive approach computes each power separately: u^2, u^3, etc. Horner's method rewrites the same polynomial as nested multiplications:

$$c_0 + u\Big(c_1 + u(c_2 + u \cdot c_3)\Big)$$

This requires only one multiplication and one addition per coefficient, working from the *highest* degree down to the constant term. The loop iterates over `reverse(p.coeffs)`, which goes from the highest-degree coefficient down to the constant.

Evaluate `Poly([1.0, 4.0, -4.0])` at $u = 0.5$ (this is $1 + 4u - 4u^2$):

`reverse([1.0, 4.0, -4.0]) = [-4.0, 4.0, 1.0]`

Iteration	c	result = result * u + c
start	—	result = 0.0
1	-4.0	0.0 * 0.5 + (-4.0) = -4.0
2	4.0	-4.0 * 0.5 + 4.0 = 2.0
3	1.0	2.0 * 0.5 + 1.0 = 2.0

Output: `2.0`

Verify: $1 + 4(0.5) - 4(0.5)^2 = 1 + 2 - 1 = 2.0$ ✓

6. The `expand_bernstein` Function and Pascal's Triangle

This is where the real algebra happens. Everything before this was setup. Now we cook.

```julia
function expand_bernstein(n::Int, i::Int)::Poly
    m = n - i
    one_minus_u = Poly([binomial(m, j) * (-1)^j for j in 0:m])
    shifted = Poly(vcat(zeros(i), one_minus_u.coeffs))
    shifted * binomial(n, i)
end
```

This function takes two integers — the degree n and the index i — and returns a `Poly` representing the Bernstein basis polynomial $B(n, i, u) = \binom{n}{i} u^i (1-u)^{n-i}$ fully expanded into monomial form.

The key ingredient is `binomial(n, k)`, Julia's built-in function for the binomial coefficient $\binom{n}{k}$. Before we see how `expand_bernstein` uses it, let us understand what it actually computes.

6.1. What is `binomial(n, k)`?

`binomial(n, k)` computes $\binom{n}{k}$, read aloud as "n choose k." It counts the number of ways to choose k items from a set of n items, without caring about order.

The formula is:

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

A few values:

`binomial(4, 0) = 1` (one way to choose nothing)
`binomial(4, 1) = 4` (four ways to choose one item from four)
`binomial(4, 2) = 6` ($\frac{4!}{2!\,2!} = \frac{24}{4} = 6$)
`binomial(4, 3) = 4`

`binomial(4, 4) = 1` (one way to choose everything)

Notice the symmetry: `binomial(4,1) == binomial(4,3)`. In general, $\binom{n}{k} = \binom{n}{n-k}$.

6.2. Pascal's Triangle — the Cheat Sheet Nobody Printed

Your teacher told you to memorise this. You didn't. Neither did we. But now it actually matters.

Pascal's triangle is a visual arrangement of all binomial coefficients. Each row n contains $\binom{n}{0}, \binom{n}{1}, \ldots, \binom{n}{n}$. Each entry is the sum of the two entries directly above it.

$$\begin{array}{lccccccccc}
n=0: & & & & & 1 & & & & \\
n=1: & & & & 1 & & 1 & & & \\
n=2: & & & 1 & & 2 & & 1 & & \\
n=3: & & 1 & & 3 & & 3 & & 1 & \\
n=4: & 1 & & 4 & & 6 & & 4 & & 1
\end{array}$$

For this program, row $n = 4$ is the one that matters (quartic case). Reading left to right: $1, 4, 6, 4, 1$.

These are precisely the prefactors of the five Bernstein basis polynomials $B(4, 0)$ through $B(4, 4)$:

$$B(4,0) = 1 \cdot (1-u)^4, \quad B(4,1) = 4 \cdot u(1-u)^3, \quad B(4,2) = 6 \cdot u^2(1-u)^2, \quad \ldots$$

The rule for building the triangle: any entry equals the sum of the two entries above it. For example, $6 = 3 + 3$, and $4 = 1 + 3$. This means you never need the factorial formula — just add neighbours. Of course, Julia's `binomial()` handles this for you, so you don't need to add anything by hand either.

6.3. How `expand_bernstein` Uses `binomial`

The function expands $B(n, i, u)$ in three steps. Let us trace through `expand_bernstein(4, 2)`, which should give us $B(4, 2, u) = 6u^2(1-u)^2$.

Step A — expand $(1-u)^m$ using the binomial theorem.

With $n = 4$ and $i = 2$, we have $m = n - i = 2$.

The binomial theorem says:

$$(1-u)^m = \sum_{j=0}^{m} \binom{m}{j} (-1)^j \, u^j$$

In code:

```
one_minus_u = Poly([binomial(m, j) * (-1)^j for j in 0:m])
```

This is a **list comprehension**: Julia evaluates `binomial(m,j) * (-1)^j` for each `j` from `0` to `m`, and collects the results into a vector.

For $m = 2$, iterating `j = 0, 1, 2`:

j	binomial(2,j)	(-1)^j	product
0	1	$+1$	$+1$
1	2	-1	-2
2	1	$+1$	$+1$

`one_minus_u = Poly([1.0, -2.0, 1.0])` $= 1 - 2u + u^2$

This is $(1-u)^2$. ✓

Step B — multiply by u^i by shifting coefficients.

Multiplying a polynomial by u^i shifts all its powers up by i. In coefficient-vector terms, that means prepending i zeros.

```
shifted = Poly(vcat(zeros(i), one_minus_u.coeffs))
```

`vcat` concatenates two vectors vertically (end to end).

For $i = 2$: prepend 2 zeros to `[1.0, -2.0, 1.0]`:

`vcat([0.0, 0.0], [1.0, -2.0, 1.0])` = `[0.0, 0.0, 1.0, -2.0, 1.0]`

This represents $u^2 - 2u^3 + u^4$, which is indeed $(1-u)^2$ multiplied by u^2. ✓

Step C — multiply by $\binom{n}{i}$.

```
shifted * binomial(n, i)
```

This uses the scalar `*` operator we defined earlier.

$\binom{4}{2} = 6$.

`Poly([0.0, 0.0, 1.0, -2.0, 1.0]) * 6` = `Poly([0.0, 0.0, 6.0, -12.0, 6.0])`

This is $6u^2 - 12u^3 + 6u^4$.

Which matches the output from the program:

```
B(4,2,u)  =  6u^2(1-u)^2
          =  6u^2 - 12u^3 + 6u^4
```

✓

Why does the alternating sign $(-1)^j$ appear?

The binomial theorem for $(a+b)^m$ uses $+$ signs throughout. But we have $(1 + (-u))^m$, so $b = -u$. Each power $b^j = (-u)^j = (-1)^j u^j$. That is where the alternating $+1, -1, +1, \ldots$ pattern comes from. Without it we would be expanding $(1+u)^m$ instead of $(1-u)^m$, and the curve would go somewhere completely wrong.

The sign matters. It always does.

7. The `BezierExpansion` Struct — A Concrete Map

Before looking at the print functions, it is important to know exactly what data lives inside a `BezierExpansion` object. Every print function receives one as its argument `exp`, so understanding the struct means understanding what every `exp.field` reference refers to.

```
struct BezierExpansion
    points ::Matrix{Float64}   # control points (n_pts x dim)
    axes   ::Vector{String}
    n      ::Int               # number of control points
    degree ::Int
    dim    ::Int
    basis  ::Vector{Poly}      # expanded Bernstein poly per point
    polys  ::Dict{String,Poly} # final collected poly per axis
end
```

Below is the struct filled in with the actual values for **Case 1** (quadratic 2-D, 3 control points). This is the object that gets passed to every print function.

Field	Type	Value for Case 1
`points`	`Matrix{Float64}`	$\begin{bmatrix} 1 & 1 \\ 3 & 3 \\ 5 & 1 \end{bmatrix}$ Row = one point. Column 1 = x, column 2 = y.
`axes`	`Vector{String}`	`["x", "y"]`
`n`	`Int`	3 (number of control points)
`degree`	`Int`	2 ($= n - 1$)
`dim`	`Int`	2 (number of axes)
`basis`	`Vector{Poly}`	`[Poly([1,-2,1])`, $B(2,0) = 1 - 2u + u^2$ `Poly([0,2,-2])`, $B(2,1) = 2u - 2u^2$ `Poly([0,0,1])]` $B(2,2) = u^2$
`polys`	`Dict{String,Poly}`	`"x" => Poly([1,4,0])` $= 1 + 4u$ `"y" => Poly([1,4,-4])` $= 1 + 4u - 4u^2$

Two different polynomial collections:

`basis` contains the *Bernstein* polynomials — one per control point, independent of which curve or axis you are looking at. These are the building blocks.

`polys` contains the *final* polynomial per axis, after weighting each basis polynomial by its coordinate and summing. These are what you actually evaluate to get points on the curve.

Summary: `BezierExpansion` Functions

All functions associated with `BezierExpansion`:

Function	Signature	Returns	Purpose
Constructor	`BezierExpansion(points,axes)`	`BezierExpansion`	Build the full expansion: compute all basis polys and collect per-axis polynomials
`evaluate`	`(BezierExpansion, Real)`	`Vector{Float64}`	Evaluate the curve at u; returns one coordinate per axis

The constructor does all the heavy lifting — it calls `expand_bernstein` for each control point and accumulates the weighted sums. `evaluate` is lightweight by design: it just reads the already-built `polys` dict and calls Horner on each one.

8. The Print Functions

Summary: Formatting Helper Functions

Standalone formatting utilities (not tied to any struct):

Function	Signature	Returns	Purpose
`ruler`	`(Char)`	`String`	64-character divider line
`fmt_coeff`	`(Float64)`	`String`	Integer if whole, 4 d.p. otherwise
`fmt_poly`	`(Poly, String)`	`String`	Readable polynomial string, e.g. `"1 + 4u - 4u^2"`
`fmt_bernstein_symbolic`	`(Int, Int)`	`String`	Symbolic basis string, e.g. `"6u^2(1-u)^2"`

These functions only produce strings — they never touch a `Poly` struct's internals directly, and they never print anything. Separating formatting from printing means you can reuse them anywhere.

The Five Step Functions

All five print functions receive the same `exp` struct described above, plus sometimes a list of u values. None of them return a value — they only write to the terminal.

8.1. `print_step1_bernstein_form(exp)`

What it prints: the curve formula in Bernstein form — the symbolic equation $\mathbf{P}(u) = B(n,0)P_0 + \cdots$ and the definition of each basis function.

```
function print_step1_bernstein_form(exp::BezierExpansion)
    print_step_header(1, "Bernstein Form")
    n   = exp.degree
    dim = join(["$(ax)(u)" for ax in exp.axes], "  ")
    println("  P(u) =  [$dim]")
    println()
    terms = ["B($n,$i)*P$i" for i in 0:exp.n-1]
    println("  P(u) =  " * join(terms, "  +  "))
    println()
    println("  where:")
    for i in 0:exp.n-1
        sym = fmt_bernstein_symbolic(n, i)
        println("    B($n,$i,u)  =  $sym")
    end
end
```

How it works:

1. Reads `exp.degree` (e.g. 2) and `exp.axes` (e.g. `["x","y"]`).
2. Builds the axis label string `"x(u) y(u)"` by looping over `exp.axes` and joining with spaces.
3. Builds the sum string `"B(2,0)*P0 + B(2,1)*P1 + B(2,2)*P2"` by looping `i` from `0` to `exp.n-1`.
4. Loops again over the same range and calls `fmt_bernstein_symbolic(n, i)` to produce the human-readable form like `"2u(1-u)"` for each basis function.

> Fields used: `exp.degree`, `exp.axes`, `exp.n`
>
> No actual polynomial data is used here — this step is purely symbolic, showing the formula *before* any numbers are substituted.

Actual output (Case 2):

```
STEP 1 - Bernstein Form
----------------------------------------------------------------
P(u) =  [x(u)  y(u)  z(u)]

P(u) =  B(4,0)*P0  +  B(4,1)*P1  +  B(4,2)*P2
                   +  B(4,3)*P3  +  B(4,4)*P4

where:
  B(4,0,u)  =  (1-u)^4
```

```
    B(4,1,u)  =  4u(1-u)^3
    B(4,2,u)  =  6u^2(1-u)^2
    B(4,3,u)  =  4u^3(1-u)
    B(4,4,u)  =  u^4
```

8.2. print_step2_substitution(exp)

What it prints: the actual coordinate values substituted into the Bernstein formula — one line per axis.

```julia
function print_step2_substitution(exp::BezierExpansion)
    print_step_header(2, "Substitution of Control Point Coordinates")
    n = exp.degree
    for (dim_idx, axis) in enumerate(exp.axes)
        parts = String[]
        for i in 1:exp.n
            coord = exp.points[i, dim_idx]
            sym   = fmt_bernstein_symbolic(n, i-1)
            c     = fmt_coeff(coord)
            push!(parts, "  $(sym)*$(c)")
        end
        println("  $(axis)(u) =" * join(parts, "  +  "))
    end
end
```

How it works:

The function has a **double loop**: the outer loop iterates over axes (`"x"`, `"y"`), and the inner loop iterates over control points (`i = 1` to `exp.n`).

For each combination of axis and control point, it reads `exp.points[i, dim_idx]` — the coordinate of point i on the current axis.

Reading the points matrix for Case 1:

$$\texttt{exp.points} = \begin{bmatrix} 1 & 1 \\ 3 & 3 \\ 5 & 1 \end{bmatrix}$$

i	dim_idx	axis	exp.points[i, dim_idx]	Meaning
1	1	"x"	1.0	x-coord of P_0
2	1	"x"	3.0	x-coord of P_1
3	1	"x"	5.0	x-coord of P_2
1	2	"y"	1.0	y-coord of P_0
2	2	"y"	3.0	y-coord of P_1
3	2	"y"	1.0	y-coord of P_2

Each coordinate is paired with its symbolic basis term (e.g. `"(1-u)^2*1"`) and all terms for one axis are joined into one line.

Fields used: `exp.degree`, `exp.axes`, `exp.n`, `exp.points`.

Actual output (Case 2):

```
STEP 2 - Substitution of Control Point Coordinates
---------------------------------------------------------------
x(u) =  (1-u)^4*0  +  4u(1-u)^3*0  +  6u^2(1-u)^2*4
                   +  4u^3(1-u)*4   +  u^4*5
y(u) =  (1-u)^4*0  +  4u(1-u)^3*4  +  6u^2(1-u)^2*0
                   +  4u^3(1-u)*4   +  u^4*4
z(u) =  (1-u)^4*1  +  4u(1-u)^3*1  +  6u^2(1-u)^2*1
                   +  4u^3(1-u)*1   +  u^4*1

(Each basis term will now be expanded individually)
```

Notice that the first two x terms are multiplied by 0 (because P_0 and P_1 have $x = 0$), so they will vanish in Step 4. All z terms are multiplied by 1 (every point has $z = 1$), which is why $z(u)$ will collapse to the constant 1.

8.3. `print_step3_expanded_terms(exp)`

What it prints: each Bernstein basis polynomial expanded into monomial form.

```julia
function print_step3_expanded_terms(exp::BezierExpansion)
    print_step_header(3, "Expand Each Bernstein Basis Term")
    n = exp.degree
    for i in 0:exp.n-1
        basis = exp.basis[i+1]
        sym   = fmt_bernstein_symbolic(n, i)
        ex    = fmt_poly(basis)
        println("  B($n,$i,u)  =  $sym")
        println("            =  $ex")
        println()
    end
end
```

How it works:

The loop runs `i` from `0` to `exp.n-1` (0-indexed, matching the mathematical notation $B(n, 0)$ through $B(n, n)$).

Inside the loop:

- `exp.basis[i+1]` fetches the i-th expanded basis polynomial. The `+1` is needed because Julia arrays are 1-indexed (the first element is at index 1, not 0).
- `fmt_bernstein_symbolic(n, i)` produces the symbolic string like `"2u(1-u)"`.
- `fmt_poly(basis)` converts the `Poly` coefficients into a readable string like `"2u - 2u^2"`.

> For Case 1, `exp.basis` contains three `Poly` objects:

i	exp.basis[i+1]	Printed as
0	Poly([1.0, -2.0, 1.0])	1 - 2u + u^2
1	Poly([0.0, 2.0, -2.0])	2u - 2u^2
2	Poly([0.0, 0.0, 1.0])	u^2

Fields used: `exp.degree`, `exp.n`, `exp.basis`.

Actual output (Case 2):

```
STEP 3 - Expand Each Bernstein Basis Term
----------------------------------------------------------------
B(4,0,u)  =  (1-u)^4
          =  1 - 4u + 6u^2 - 4u^3 + u^4

B(4,1,u)  =  4u(1-u)^3
          =  4u - 12u^2 + 12u^3 - 4u^4

B(4,2,u)  =  6u^2(1-u)^2
          =  6u^2 - 12u^3 + 6u^4

B(4,3,u)  =  4u^3(1-u)
          =  4u^3 - 4u^4

B(4,4,u)  =  u^4
          =  u^4
```

Each pair of lines is one iteration of the loop: the first line comes from `fmt_bernstein_symbolic`, the second from `fmt_poly(exp.basis[i+1])`. The `exp.basis` vector holds these five `Poly` objects (one per control point), already computed by `expand_bernstein` during construction.

8.4. `print_step4_collected(exp)`

What it prints: for each axis, shows each weighted contribution (`basis[i] * coord`), then the final collected polynomial.

```julia
function print_step4_collected(exp::BezierExpansion)
    print_step_header(4, "Collect Terms - Final Monomial Polynomial")
    n = exp.degree
    for (dim_idx, axis) in enumerate(exp.axes)
        println("  $(axis)(u) - weighted contributions:")
        for i in 1:exp.n
            coord   = exp.points[i, dim_idx]
            term    = exp.basis[i] * coord
            exp_str = fmt_poly(term)
            c_str   = fmt_coeff(coord)
            println("    B($n,$(i-1))*$(lpad(c_str,4))  =  $exp_str")
        end
        println()
    end
    println("  Collected:")
    for axis in exp.axes
```

```
        poly = exp.polys[axis]
        println("    $(axis)(u)  =  $(fmt_poly(poly))")
    end
end
```

How it works:

Again a double loop: outer over axes, inner over control points.

Inside the inner loop:

- `exp.points[i, dim_idx]` reads the coordinate (same as Step 2).
- `exp.basis[i] * coord` multiplies the expanded basis polynomial by that coordinate using the `*` override we defined earlier. This produces a new `Poly` — the weighted contribution.
- `fmt_poly(term)` converts that `Poly` to a string for printing.
- `lpad(c_str, 4)` left-pads the coordinate string to 4 characters so the columns line up neatly.

After both loops, a second loop over `exp.axes` prints the *final* collected polynomial from `exp.polys[axis]`. This is the result that was computed once during construction and stored in the struct — the print function just reads it.

For the x axis in Case 1:

i	coord	basis[i] * coord	Printed
1	1.0	`Poly([1,-2,1])*1.0`	`1 - 2u + u^2`
2	3.0	`Poly([0,2,-2])*3.0`	`6u - 6u^2`
3	5.0	`Poly([0,0,1])*5.0`	`5u^2`

Then: `exp.polys["x"] = Poly([1.0, 4.0, 0.0])` is printed as `1 + 4u`.

Fields used: `exp.degree`, `exp.axes`, `exp.n`, `exp.points`, `exp.basis`, `exp.polys`.

Actual output (Case 2):

```
STEP 4 - Collect Terms - Final Monomial Polynomial
---------------------------------------------------------------
x(u) - weighted contributions:
  B(4,0)*   0  =  0
  B(4,1)*   0  =  0
  B(4,2)*   4  =  24u^2 - 48u^3 + 24u^4
  B(4,3)*   4  =  16u^3 - 16u^4
  B(4,4)*   5  =  5u^4

y(u) - weighted contributions:
  B(4,0)*   0  =  0
  B(4,1)*   4  =  16u - 48u^2 + 48u^3 - 16u^4
  B(4,2)*   0  =  0
  B(4,3)*   4  =  16u^3 - 16u^4
  B(4,4)*   4  =  4u^4
```

```
z(u) - weighted contributions:
  B(4,0)*   1  =  1 - 4u + 6u^2 - 4u^3 + u^4
  B(4,1)*   1  =  4u - 12u^2 + 12u^3 - 4u^4
  B(4,2)*   1  =  6u^2 - 12u^3 + 6u^4
  B(4,3)*   1  =  4u^3 - 4u^4
  B(4,4)*   1  =  u^4

Collected:
  x(u)  =  24u^2 - 32u^3 + 13u^4
  y(u)  =  16u - 48u^2 + 64u^3 - 28u^4
  z(u)  =  1
```

The inner loop produces one row per control point per axis. The `lpad(c_str, 4)` call right-aligns the coordinate (e.g. `" 0"`, `" 4"`, `" 5"`) so the equals signs line up. After the inner loop ends, `exp.polys[axis]` is read directly for the "Collected" lines — that value was computed once in the constructor and is just retrieved here.

8.5. print_step5_evaluation(exp, u_values)

What it prints: a table of curve points at each u value, then a detailed substitution check at the middle u.

```julia
function print_step5_evaluation(exp::BezierExpansion,
                                u_values::Vector{Float64})
    print_step_header(5, "Evaluation at Given u Values")
    axes  = exp.axes
    dim   = length(axes)
    col_u = 6
    col_v = 10

    header = "  " * lpad("u", col_u)
    for ax in axes
        header *= "  " * lpad("$(ax)(u)", col_v)
    end
    println(header)
    println("  " * "-"^(col_u + (col_v + 2) * dim + 2))

    for u in u_values
        pt  = evaluate(exp, u)
        row = "  " * lpad(@sprintf("%.2f", u), col_u)
        for v in pt
            row *= "  " * lpad(@sprintf("%.6f", v), col_v)
        end
        println(row)
    end
    ...
end
```

How it works:

The header row. `lpad("u", 6)` pads the string `"u"` to width 6 by adding spaces on the left. The same is done for each axis label. This creates aligned columns.

The data rows. For each `u` in `u_values`, it calls `evaluate(exp, u)`, which returns a vector with one value per axis (e.g. `[3.0, 2.0]` for x and y). Each value is formatted to 6 decimal places with `@sprintf` and padded to width 10 with `lpad`.

The verification block. After the table, the code picks the middle `u`:

```
u_check = u_values[div(length(u_values), 2) + 1]
```

`div` is integer division (no remainder). For 5 values: `div(5, 2) = 2`, so `u_values[3] = 0.50`.

Then for each axis it loops over the polynomial's coefficients and prints each term as a separate substitution (e.g. `"4*0.50^1"`), so you can follow the arithmetic by hand.

Fields and arguments used:

What	Where it comes from
Axis labels in header	`exp.axes`
Curve values in rows	`evaluate(exp, u)` → `exp.polys`
Middle u pick	`u_values` (the external argument)
Substitution terms	`exp.polys[axis].coeffs` looped directly

Actual output (Case 2):

```
STEP 5 - Evaluation at Given u Values
-------------------------------------------------------------
     u        x(u)        y(u)        z(u)
---------------------------------------------
  0.00    0.000000    0.000000    1.000000
  0.25    1.050781    1.890625    1.000000
  0.50    2.812500    2.250000    1.000000
  0.75    4.113281    3.140625    1.000000
  1.00    5.000000    4.000000    1.000000

Verification at u = 0.50:
  x(0.50)  =  24u^2 - 32u^3 + 13u^4
            =  24*0.50^2  +  -32*0.50^3  +  13*0.50^4
            =  2.812500

  y(0.50)  =  16u - 48u^2 + 64u^3 - 28u^4
            =  16*0.50^1  +  -48*0.50^2  +  64*0.50^3  +  -28*0.50^4
            =  2.250000

  z(0.50)  =  1
            =  1
            =  1.000000
```

The table is built column by column: `u` is printed first (width 6), then each axis value (width 10), all left-padded so decimal points align. The verification block loops over `poly.coeffs` directly, skipping any coefficient whose absolute value is below 10^{-9} (same threshold used in `fmt_poly`). For $z(u) = 1$ there is only a constant term, so the substitution line just shows `"= 1"` with no powers of u.

Summary: Print Functions

All print functions (none return a value — output only):

Function	Signature	Purpose
`print_step_header`	`(Int, String)`	Print numbered step heading with dashed rule
`print_title`	`(BezierExpansion, Int)`	Print the title banner with degree and dimension info
`print_control_points`	`(BezierExpansion)`	List all control points with coordinates
`print_step1_bernstein_form`	`(BezierExpansion)`	Show symbolic Bernstein formula and basis definitions
`print_step2_substitution`	`(BezierExpansion)`	Show coordinates substituted into basis terms
`print_step3_expanded_terms`	`(BezierExpansion)`	Show each basis polynomial in monomial form
`print_step4_collected`	`(BezierExpansion)`	Show weighted contributions and final collected polynomial
`print_step5_evaluation`	`(BezierExpansion, Vector{Float64})`	Print evaluation table and mid-point verification
`print_all`	`(BezierExpansion, Vector{Float64}, Int)`	Orchestrate steps 1–5 in order

`print_all` is the conductor. It calls everything in sequence and adds nothing of its own. If you ever want to print just one step in isolation — say, only the evaluation table — you can call `print_step5` directly. The steps are independent.

9. Summary: How Data Flows Through the Program

Step	Function	Data used from `exp`
1	`print_step1`	`degree`, `axes`, `n` — symbolic only, no numbers
2	`print_step2`	`degree`, `axes`, `n`, `points` — coordinates substituted
3	`print_step3`	`degree`, `n`, `basis` — expanded basis polys printed
4	`print_step4`	`degree`, `axes`, `n`, `points`, `basis`, `polys` — weighted terms and final result
5	`print_step5`	`axes`, `polys` — final polys evaluated at each u

Code explanation for `01-bezier-algebra.jl` — output examples use Case 2 | Prepared by E.R. Nurwijayadi | Prompt-driven, AI-composed